# RPC: Remote Procedure Call

## Sistemas Operativos I

Erica Vidal

2026

# De Socket a Microservicios

La filosofía detrás de RPC (Remote Procedure Call) es hacer que la ejecución de una función en una máquina remota se vea y se escriba exactamente igual que la llamada a una función local.

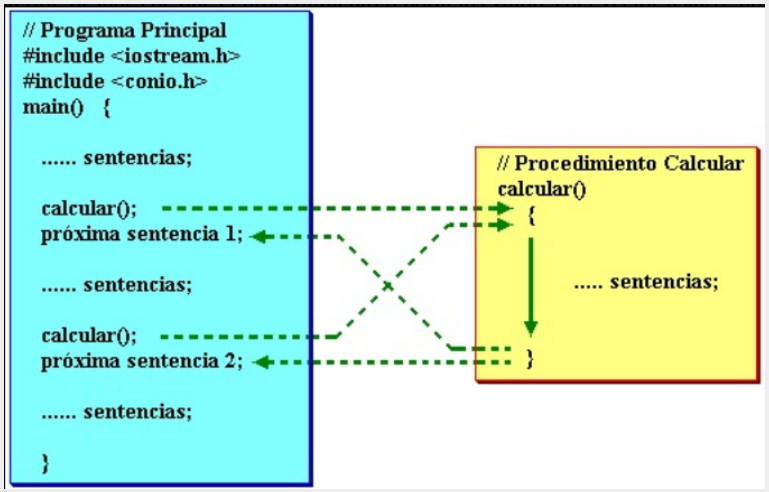
# Un poco de historia

## Algunos hitos:

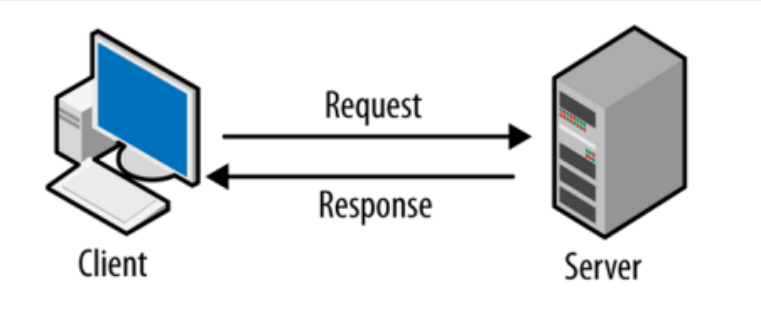
- 1981: Bruce Jay Nelson publica su tesis en Xerox PARC definiendo RPC. Su meta era la transparencia total de red.
- 1984 (Sun RPC): La usaron para construir NFS.
- Años 90 : Aparecen CORBA, DCOM y Java RMI. Intentaron llevar RPC a la programación orientada a objetos (objetos distribuidos).
- Años 2000: Web: SOAP (RPC sobre XML), luego REST.
- 2007 (Apache Thrift): Facebook. Vuelve el RPC por necesidad de velocidad. Binarios en lugar de texto.
- 2015 (gRPC): Google libera gRPC. Se convierte en el estándar actual. Usa HTTP/2 y Protocol Buffers.
- 2020s tRPC: RPC al dev. web moderno (Next.js/React).

# Imaginar los siguientes casos

a)




b)




RPC trata de hacer que un esquema como b) sea visto por el programador lo más parecido a a).

# ¿Cómo lo hace?

## Generador automático de documentación

- **IDL (Interface Definition Language)**: Un lenguaje neutro donde declaramos las funciones y los tipos.
- **Generador de Stubs (rpcgen)**: Un compilador que lee el IDL y genera código C automáticamente (el código de los sockets).
- **Client Stub:** Una función generada automáticamente que "finge" ser la función real en el cliente. Su trabajo es empaquetar los parámetros (Marshalling) y mandarlos por la red.
- **Server Stub:** El código generado en el servidor que recibe el paquete, lo desempaqueta (Unmarshalling) e invoca la verdadera función.

# Ejemplo

## Cada servicio se diseña como un prototipo de una función

```
/* calculadora.x */
struct operandos {
    int a;
    int b;
};
program CALCULADORA_PROG {
    version CALCULADORA_VERS {
        int SUMAR(operandos) = 1;
    } = 1;
} = 0x20000001;
/* Número de programa único (asignado arbitrariamente)
```

Los números ya usados están en /etc/services

# Ejemplo

## Instalar dependencias

En versiones modernas de Ubuntu, las herramientas de Sun RPC están en paquetes separados:

```
sudo apt update
sudo apt install rpcbind rpcsvc-proto \
  build-essential libtirpc-dev
```

# Ejemplo

**Es parecido a C pero no es C.**

Para implementar el protocolo se usa (en bash):

- `rpcgen -C calculadora.x`

**Si todo anda bien vamos a tener además de calculadora.x a**

`calculadora_clnt.c`, `calculadora_xdr.c`, `calculadora_svc.c`, `calculadora.h`.

# Ejemplo

Crear `server.c`..El prototipo de la función nos la impone el archivo .h generado.

C

```
#include "calculadora.h"
#include <stdio.h>

/* ONC RPC requiere devolver un puntero */
int * sumar_1_svc(operandos *argp, \
            struct svc_req *rqstp) {
    static int resultado;
    printf("Servidor: sumar %d + %d\n", \
                    argp->a, argp->b);
    resultado = argp->a + argp->b;
    return &resultado;}
```

# Ejemplo

Crear `client.c`..

C

```
#include "calculadora.h"
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    CLIENT *clnt;
    int *resultado;
    operandos args;
    if (argc != 4) {
        printf("Uso: %s <ip_servidor> <num1> \
            <num2>\n", argv[0]);
        exit(1);}
```

# Ejemplo

Crear `client.c`. (cont).

C

```
/* 1. Creamos el handle de red
(internamente abre un socket UDP/TCP) */
clnt = clnt_create(argv[1], CALCULADORA_PROG,\
      CALCULADORA_VERS, "udp");
if (clnt == NULL) {
    clnt_pcreateerror(argv[1]);
    exit(1);
}

/* 2. Preparamos datos */
args.a = atoi(argv[2]);
args.b = atoi(argv[3]);
```

# Ejemplo

Crear `client.c`. (cont).

C

```
/* 3. ¡LA LLAMADA RPC! Parece una función local */
 resultado = sumar_1(&args, clnt);

 if (resultado == NULL) {
     clnt_perror(clnt, "Fallo en la llamada RPC");
 } else {
     printf("El resultado remoto es: %d\n", \
            *resultado);
 }

 clnt_destroy(clnt);
 return 0;}
```

# Ejemplo

**Para compilar:**

```
gcc -o cliente client.c calculadora_clnt.c \
      calculadora_xdr.c -I/usr/include/tirpc -ltirpc

gcc -o servidor server.c calculadora_svc.c \
    calculadora_xdr.c -I/usr/include/tirpc -ltirpc
```

**Para ejecutar:**

Abrir dos terminales:
En una ejecutar ./servidor.
En la otra: ./cliente localhost 15 25

# Notas finales

¿Se usa rpcgen hoy? No para sistemas web modernos. Pero el paradigma RPC ha regresado con fuerza bajo el nombre de gRPC (creado por Google).
En la arquitectura moderna de Microservicios, REST se volvió muy ineficiente (parsear JSON consume mucha CPU). gRPC vuelve a las raíces de rpcgen

- El IDL: En lugar de .x, usamos Protocol Buffers (.proto).
- La codificación (XDR): Se reemplaza por binarios Protobuf ultracompactos.
- El Transporte (UDP/TCP): Se reemplaza por HTTP/2, que permite multiplexar múltiples llamadas RPC en un solo socket TCP.

# Para concluir

## ¿por qué la premisa original de Nelson de 1981 falló?

- Una llamada de red NUNCA es igual a una llamada local.
  - Si sumar() es local y falla, el programa crashea (fácil de debuggear).
  - Si sumar_1() remoto falla (retorna NULL), ¿qué pasó? ¿Murió el servidor? ¿Se cortó el cable de red? ¿El servidor la calculó pero la respuesta se perdió a la vuelta?

RPC moderno no intenta ocultar la red; herramientas como gRPC nos obligan a programar asumiendo que la red va a fallar, obligándonos a definir timeouts y lógicas de reintento.